

Discussion Preparation Guide — EGX Multi-Agent Stock Prediction System

Topics: Social Media Sentiment Analysis · Agents Memory · Reinforcement Learning · Database Layer · Portfolio Manager Agent

Context: Graduation project defense. Examiner is strong technically. Be ready for deep "why" questions.

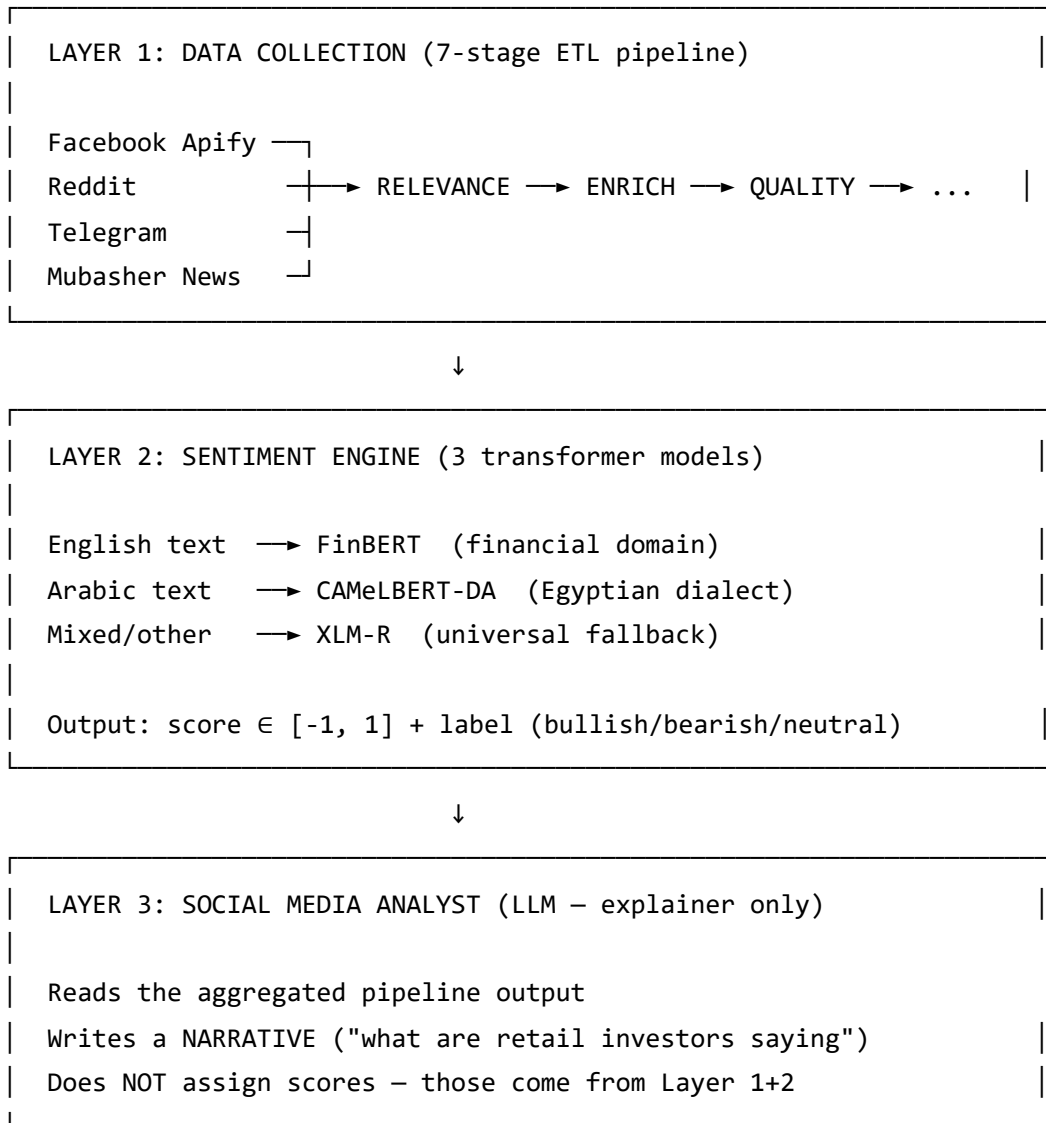
TABLE OF CONTENTS

1. [Social Media Sentiment Analysis](#)
2. [Why Both Agent + Engine? \(Key Question\)](#)
3. [Agents Memory System](#)
4. [Reinforcement Learning Meta-Policy](#)
5. [Database Layer](#)
6. [Portfolio Manager Agent](#)
7. [Master Q&A — Hard Technical Questions](#)

1. Social Media Sentiment Analysis

1.1 — The Big Picture (say this first)

"Our social media pipeline has **three separate layers** that work together. No single layer can do the job alone."



1.2 — The 7-Stage v2 Pipeline (Layer 1)

File: tradingagents/dataflows/social_v2/pipeline.py

STAGE 1: SCRAPE

- Facebook Apify (paid actor, 5 curated Arabic EGX groups)
- Reddit (intent-based queries)
- Telegram (public t.me/s/ previews)
- Mubasher News (free EGX RSS feed)

STAGE 2: RELEVANCE GATE

- 2-signal requirement: finance context AND EGX mention
- Exception: Facebook/Telegram bypass (the group IS the EGX signal)
- Reddit is STRICT: must mention a real ticker symbol

Why? Reddit full-text search returns off-topic posts.

Facebook posts come from curated EGX investment groups.

STAGE 3: ENRICH

- Entity extraction: which EGX tickers are mentioned? (84 in registry)
 - Intent detection: BUY/SELL/BULLISH/BEARISH/HOLD/REACTION
 - Content type: OPINION / NEWS / ANALYSIS / QUESTION
- Arabic patterns: اشترى (BUY), بيع (SELL), هيطلع (BULLISH), هينهار (BEARISH)

STAGE 4: QUALITY GATE (permissive)

- DROP: empty, <3 words, >150 words, pure questions
 - DOWNGRADE (keep, lower weight): non-actionable content
- Why permissive? EGX social data is sparse. Hard drops lose signal.

STAGE 5: SENTIMENT

- SentimentEngine.analyze_batch() – transformer models
- VADER baseline (for comparison)
- Both scores stored in each post

STAGE 6: AGGREGATE

- Weighted per-stock formula:
$$\text{weight} = \text{content_weight} \times \text{intent_factor} \times \text{platform_boost}$$
$$\text{engagement} = \log(1 + \text{likes} + \text{comments} + \text{shares}) \leftarrow \text{log dampens outliers}$$
$$\text{final_score} = \Sigma(\text{sentiment} \times \text{weight} \times \text{engagement}) / \Sigma(\text{weight} \times \text{engagement})$$
$$\text{confidence} = 0.6 \times \text{quality} + 0.4 \times (n / (n + 5))$$

STAGE 7: ARCHIVE

- Write to Postgres social_v2_posts table
- For backtest replay of REAL historical social data

Content Type Weights:

OPINION	→ 1.0	("I think COMI will rally")	– Full weight
NEWS	→ 0.6	("Company reported earnings")	– Lower (factual not sentiment)
ANALYSIS	→ 0.4	("RSI is oversold at 30")	– Reduced (analytical)
QUESTION	→ 0.2	("Should I buy COMI?")	– Minimal

Why different weights? A retail trader's emotional opinion carries more sentiment signal than a factual news headline.

1.3 — The Sentiment Engine (Layer 2)

File: tradingagents/utils/sentiment_engine.py

Why 3 models instead of 1?

Model	Language	Why This Model
FinBERT (ProsusAI/finbert)	English	Pre-trained on financial text — earnings calls, analyst reports. Understands "beat estimates", "guidance cut" better than generic BERT
CAMeLBERT-DA (CAMEL-Lab/bert-base-arabic-camelbert-da-sentiment)	Arabic (MSA + Egyptian dialect)	Generic models fail on Egyptian colloquial: "صاروخ" (rocket = mooning), "هيطير" (will fly = breakout). This model was

Model	Language	Why This Model
		trained on dialectal Arabic
XLM-R (cardiffnlp/twitter-xlm-roberta-base-sentiment-multilingual)	Mixed / fallback	Handles code-switching (Arabic + English in one post). Trained on 100+ languages

Routing logic:

```

if language == "en": return "finbert"
elif language == "ar": return "camelbert"
else: return "xlmr" # fallback

```

Fallback chain:

```

FinBERT loads OK? → use FinBERT
FinBERT fails? → try XLM-R
XLM-R fails? → rule-based lexicon (500+ Arabic + English terms)
               but confidence capped at 0.45 to signal "weaker"

```

Negation handling (Arabic):

"مش هيرتفع" → NOT going up
The system checks 20 chars before the word for negation terms
Negated score: $-\text{score} \times 0.7$ (weakened, not fully inverted)

Hype detection:

Flag if 2+ of:

- ≥ 15 posts about same stock (unusual for EGX)
- ≥ 5 rocket/fire emojis 🚀🔥
- ≥ 3 extreme phrases ("صاروخ", "moon", "فرصة العمر")

Result: confidence reduced (prevents chasing hype)

1.4 — The Social Media Analyst Agent (Layer 3)

File: tradingagents/agents/analysts/social_media_analyst.py

Critical design decision (Phase 3 redesign):

OLD design: LLM outputs sentiment_score + direction + confidence number

NEW design: LLM outputs NARRATIVE ONLY

Directional scores come from the DETERMINISTIC pipeline

What the agent actually does:

1. Reads the pre-computed pipeline output from AgentState (fast — no new API calls)
2. Writes 2-3 sentences: "What are retail investors saying? Key themes? Arabic signals?"
3. Cites specific post IDs to back up claims
4. Does NOT output any score. Any score fields the LLM tries to add are STRIPPED.

Why this redesign?

- LLMs hallucinate confidence numbers (say "0.85" when they have no basis)
- LLMs are inconsistent in $[-1, 1]$ scaling across runs
- Deterministic pipeline gives the same score every time (auditable)
- LLM's real strength is narrative synthesis, not arithmetic

The Blend (what the agent DOES compute):

The agent computes multipliers (not direction) from market/macro/sector sentiment:

Market regime EUPHORIA? → confidence × 0.70, position size × 0.60
Market regime PANIC? → confidence × 0.70, position size × 0.50
Market regime NEUTRAL? → confidence × 1.00, position size × 1.00 (pass-through)

Example:

Analyst says: BUY, confidence 0.80
Market is in EUPHORIA (hype alert)
Blended: BUY, confidence $0.80 \times 0.70 = 0.56$ → smaller position

The blend NEVER changes direction (BUY stays BUY). It only scales confidence and size.

2. Why Both Agent + Engine? The Core Question

The examiner WILL ask: "Why not just use the LLM agent directly? Why the separate sentiment engine?"

The Answer (memorize this structure):

Layer	What it does	Why necessary
Sentiment Engine (transformer)	Token-level scoring Same post → same score always	Reproducible, auditable, no drift
v2 Pipeline (deterministic)	Multi-post aggregation Weighting, gates, hype detection	Deterministic math. No LLM needed.
LLM Agent (explainer)	Narrative synthesis "What are investors saying?" Cites posts. No scoring.	Human-readable context for the bull/bear researchers

The one-sentence answer:

"The sentiment engine handles the 'what score' question deterministically and reproducibly. The agent handles the 'what does it mean' question in human language. Mixing them would make the score non-reproducible."

If asked: could you replace the engine with GPT-4 sentiment?

"GPT-4 might score the same post differently on two runs. In a trading system, non-reproducibility means you can't audit why you made a trade. Our pipeline produces the same score for the same post every time — which is required for a defensible audit trail."

If asked: why not just one model (FinBERT only)?

"EGX social media is predominantly Arabic, often in Egyptian dialect. FinBERT was trained on English financial text. It fails on Arabic and would miss the majority of our data. CAMELBERT was specifically trained on dialectal Arabic including Egyptian colloquial."

3. Agents Memory System

3.1 — What the Memory Does

File: tradingagents/agents/utils/memory.py (class FinancialSituationMemory)

Think of it like this: after each trade, the system writes a "lesson learned" into a database. The next time it analyzes the same stock, it retrieves relevant past lessons and injects them into the agent's prompt.

WRITE PATH (after trade):

Outcome known → "COMI.CA rallied 8% after central bank rate cut"
→ Embed as vector → Store in ChromaDB with metadata {ticker, date, type}

READ PATH (before next trade):

Agent builds current situation description
→ "COMI.CA showing breakout, rate cut expected next week"
→ Similarity search against ChromaDB
→ Retrieve top-2 most similar past situations
→ Inject into prompt: "## Lessons from Past Trades"

5 separate collections (one per agent role):

Collection	Agent	What it remembers
bull_memory	Bull Researcher	Previous bullish theses that proved right/wrong

Collection	Agent	What it remembers
bear_memory	Bear Researcher	Previous bearish theses
trader_memory	Trader	Execution lessons (slippage, liquidity issues)
invest_judge_memory	Research Manager	Debate outcomes
risk_manager_memory	Risk Manager	Past risk decisions

3.2 — Dual Backend (ChromaDB vs BM25)

```
LLM Provider has embedding API?  
(OpenAI: text-embedding-3-small / Ollama: nomic-embed-text)  
  YES → ChromaDB vector search (cosine similarity ≥ 0.30)  
  NO  → BM25 keyword search (Okapi weighting)  
        Score normalized via tanh(score/5.0) → same [0,1] scale  
        So min_similarity threshold works identically in both cases
```

Why BM25 fallback?

"DeepSeek and Groq don't have embedding APIs. If we required vector search, the system would break with those providers. BM25 works offline with any LLM."

3.3 — Cold Start: Seed Corpus

On first run, ChromaDB is empty. System bootstraps with 41 hand-curated EGX precedents:

- 12 bullish scenarios
- 10 bearish scenarios
- 8 execution lessons
- 6 investment judgments
- 5 risk decisions

These are real EGX scenarios (e.g., "[COMI.CA](#) sector-wide correction during political uncertainty") that give agents a baseline even on day 1.

3.4 — Audit Logger (bonus point to mention)

Every session also writes a structured audit trail:

- `audit_log.jsonl` — append-only machine-readable log

- `audit_summary.md` — human-readable markdown

Logged events: `AGENT_OPINION` , `TRADE_JUSTIFICATION` , `RISK_DECISION` , `FINAL_ACTION`

Why audit?

"For a trading system, reproducibility and accountability are non-negotiable. If we're ever asked 'why did you buy [COMI.CA](#) on May 15?', we can replay exactly what each agent said, what confidence scores were assigned, and whether the risk manager approved."

3.5 — PostgreSQL Alternative (pgvector)

Optional swap: set `TRADINGAGENTS_MEMORY_BACKEND=postgres` → uses pgvector with IVFFlat index for cosine similarity. Same API as ChromaDB. Falls back to ChromaDB automatically if pgvector unavailable.

4. Reinforcement Learning Meta-Policy

4.1 — What Problem RL Solves

The LLM analysts say "BUY" but don't know how large a position to take given current market regime. RL learns this from historical outcomes.

Without RL:

Analysts say BUY with 80% confidence → always use 80% of max position

With RL:

Analysts say BUY with 80% confidence

RL checks: "Is this market regime/volatility/context one where 80% confidence historically meant good outcomes?"

RL output: `size_multiplier = 0.50` → use only 40% of max position

Key constraint: RL can only REDUCE positions, never GROW them. The LLM recommendation is the ceiling.

4.2 — Algorithm: Conservative Q-Learning (CQL)

Files: `tradingagents/rl/policy.py` , `tradingagents/rl/train.py`

Why offline RL?

We can't run a live trading bot to collect experience. We have a fixed historical dataset of (situation, decision, outcome) triples. This is the classic offline RL setting.

Why CQL specifically?

With ~1,500 EGX samples, standard Q-learning would extrapolate wildly and assign high Q-values to untested (state, action) pairs. CQL computes a LOWER BOUND on Q-values — it says "don't be confident about actions you've never seen." This is critical for small-data offline RL.

```
Loss = TD_loss +  $\alpha$  × CQL_penalty
      = MSE(Q(s, a_observed), reward)
      +  $\alpha$  × [log  $\sum_a \exp(Q(s,a)/\tau)$  - Q(s, a_observed)]
                ↑
                penalize high Q on unobserved actions
```

Why single-step (bandit) not multi-step MDP?

The LLM outputs one sizing recommendation. We have one decision point, one 20-day outcome. There's no sequential trajectory to exploit. Single-step CQL is theoretically correct for this setting.

4.3 — The System Design

Action Space (5 discrete tiers):

```
Action 0 → size_multiplier = 0.00 (no position)
Action 1 → size_multiplier = 0.25 (25% of recommended position)
Action 2 → size_multiplier = 0.50
Action 3 → size_multiplier = 0.75
Action 4 → size_multiplier = 1.00 (full recommendation)
```

State Space (~50 features):

Analyst confidences (3):	technical_conf, fundamental_conf, sentiment_conf
Market regime (6 one-hot):	EUPHORIA, GREED, NEUTRAL, FEAR, PANIC, NO_SIGNAL
Volatility mood (4 one-hot):	CALM, ELEVATED, STRESSED, NO_SIGNAL
Macro direction (4 one-hot):	RISK_ON, NEUTRAL, RISK_OFF, NO_SIGNAL
Sector (7 one-hot):	Banks, Real Estate, Industry, Telecom, FinSvc, Food, Unknown
Portfolio context (5):	drawdown, cash_pct, unrealised_pnl, days_held, avg_volume
Blend signals (2):	confidence_multiplier, position_size_multiplier

Network Architecture:

Input (50 features) → Linear(64) → ReLU → Dropout(0.1)
→ Linear(64) → ReLU → Dropout(0.1)
→ Linear(5) (Q-values for each action)
~5,000 parameters – intentionally tiny for 1,500 training samples

Reward Function:

```
reward = forward_return_20d
        - 0.5 × max_drawdown_during_hold (penalize volatile paths)
        - 0.00378 (EGX transaction costs: 0.189% × 2 sides)
clipped to [-0.5, 0.5]
```

Training:

- Time-aware split: per-ticker chronological tail = validation (20%)
- 200 epochs max, early stopping (30 epoch patience)
- Adam optimizer, lr=3e-4, weight_decay=1e-4
- Gradient clipping at 1.0 norm
- Fail-closed: if no model loaded → return size_multiplier=1.0 (identity)

Walk-Forward Evaluation:

PASS criteria:

- RL arm Sharpe $\geq$ baseline Sharpe + 0.20
- AND 95% bootstrap CI on per-day returns does NOT overlap zero

Risk-free rate: 24% (Egyptian CBE policy rate, 2024-2026)
NOT 5% – using wrong risk-free rate would overstate Sharpe

4.4 — Backtest Integrity (crucial)

Look-ahead protection:

All forward returns are computed from raw OHLCV data at training time, NOT from any `trade_result` field that might contain future knowledge. `trade_date < forward_window_start_date` is asserted for every record.

5. Database Layer

5.1 — Polyglot Persistence (5 stores)

The system uses **5 different storage layers**. Each is chosen for a specific reason:

PostgreSQL	→ Audit trail + backtest history
Why: ACID transactions, SQL queries, durable, ~6,600 rows/year	
ChromaDB	→ Agent memory (vector search)
Why: Cosine similarity over embeddings, SQLite-backed, <50MB	
Redis	→ Real-time WebSocket event streaming
Why: Sub-millisecond pub/sub, volatile OK (stream dies = fine)	
DiskCache	→ TTL cache for vendor API responses
Why: LRU SQLite, prevents re-fetching same news/price data	
BM25 corpus	→ Keyword fallback for memory (in-memory)
Why: Works without embedding API (DeepSeek/Groq have no embed.)	

5.2 — PostgreSQL Schema (key tables)

analysis_sessions — One row per trade decision:

```
session_id      TEXT UNIQUE  -- "COMI.CA_20260515_143022"
ticker          TEXT        -- "COMI.CA"
trade_date      DATE        -- Analysis date
final_decision  TEXT        -- "BUY" / "SELL" / "HOLD"
risk_veto       BOOLEAN     -- Was it vetoed?
confidence_overall NUMERIC(5,3) -- 0.000 to 1.000
execution_plan  JSONB       -- Position sizing details
model_fingerprint JSONB     -- {provider, model, temperature, seed}
full_state      JSONB       -- Complete graph.invoke() output
```

agent_events — Up to 13 rows per session, one per agent:

```

session_id      → links to analysis_sessions
agent_name      -- "bull_researcher", "risk_manager", etc.
event_type      -- "analyst_output", "debate", "risk_judge", "final_decision"
opinion_type    -- "bullish" / "bearish" / "neutral"
confidence_score NUMERIC(5,3)
structured_output JSONB      -- Full JSON from agent

```

backtest_runs — One row per backtest run:

```

run_id, ticker, strategy (llm / classical), start_date, end_date,
total_return_pct, sharpe_ratio, calmar_ratio, max_drawdown_pct, win_rate_pct

```

social_v2_posts — Social media archive for backtest replay:

```

post_hash TEXT UNIQUE  -- SHA256 (idempotent: re-scraping is safe)
platform, text, engagement
symbols TEXT[]          -- GIN-indexed array of tickers mentioned
intents TEXT[]
sentiment_score REAL    -- [-1.0, 1.0]

```

5.3 — Data Flow: One Analysis Run

1. Prefetch runs (parallel):
DiskCache → news/OHLCV cached? Yes → use cached. No → fetch + cache.
2. Agents run:
ChromaDB → memory retrieval for each agent
Redis → publish agent_started / agent_finished events
Dashboard WebSocket → shows live progress to user
3. Final decision:
Postgres → INSERT analysis_sessions + agent_events (audit row)
4. Post-trade (async):
Postgres → UPDATE social_v2_posts (if archiving new posts)
ChromaDB → add_situations() (reflection memory update)

5.4 — Why Not Just One Database?

Need	Wrong choice	Right choice
Vector similarity search	PostgreSQL text search	ChromaDB / pgvector
Sub-ms pub/sub streaming	PostgreSQL LISTEN/NOTIFY	Redis
TTL-expiring API cache	Redis (memory waste)	DiskCache (disk-backed)
Durable structured audit	ChromaDB	PostgreSQL

The one-sentence answer:

"Each store is optimized for a specific access pattern. Using PostgreSQL for everything would work, but vector search, pub/sub, and TTL caching would be orders of magnitude slower."

5.5 — Graceful Degradation

All stores except DiskCache are optional:

- No Postgres → audit logs skipped (warn), backtest save skipped (warn)
- No Redis → WebSocket streaming disabled, analysis still runs
- No Chroma → BM25 fallback, agents still work (less semantic precision)
- No DiskCache → vendor APIs called every time (slower, costs more)

6. Portfolio Manager Agent

6.1 — The Three-Layer Risk Pipeline

TRADER AGENT

↓ produces execution plan (position sizing, entry/exit logic)

RISK SCORER (deterministic – runs FIRST)

├ VETO? → Stop. Return HOLD. No LLM called.

└ ALLOW/WARN/THROTTLE → continue

MERGED RISK DEBATE (3 LLM agents)

├ Risky debater: "here's why this is a problem"

├ Safe debater: "here's the mitigation"

└ Neutral debater: synthesis

RISK MANAGER (LLM judge + Constitution + Final Deterministic Gate)

↓

final_trade_decision

Why this order (deterministic FIRST)?

"Putting the hard rules first is equivalent to SEC Rule 15c3-5 risk management — you can't let an LLM debate override a hard regulatory limit. The LLM layers add nuance ONLY after the hard rules pass."

6.2 — The Trader Agent

File: tradingagents/agents/trader/trader.py

What decisions it makes:

1. Position sizing (shares) based on ADV and portfolio value
2. Entry logic (limit price zone, order type)
3. Exit logic (3-tranche take-profit, stop-loss)
4. Timing (spread over multiple days if large position)

EGX Constraints injected into every prompt:

- Daily Price Limit: $\pm 10\%$ (circuit breaker)
- Trading Hours: 10:00-14:30 Cairo Time (4.5 hours only)
- Settlement: T+2 via MCDR
- ALLOWED order types: limit, limit_ioc, VWAP, TWAP
- FORBIDDEN order types: market, market_on_close, stop_market
- Max ADV: 10% per day (halved to 5% for low-liquidity stocks)
- Max portfolio concentration: 10% per stock (live) / 100% (backtest)

Memory integration:

```
past_memories = memory.get_memories(
    current_situation,
    n_matches=2,
    where={"ticker": ticker},
    min_similarity=0.30,
)
# → Injected as "## Lessons from Past Trades" in the trader's prompt
```

Structured output (JSON):

```
{
  "execution_plan": {
    "decision": "BUY",
    "position_sizing": {
      "target_shares": 50000,
      "max_shares_per_day": 15000,
      "execution_days": 4
    },
  },
  "entry_logic": { "order_type": "vwap", "limit_price": 45.50 },
  "exit_logic": {
    "take_profit": { "target_1": {"price": 50.00, "pct_of_position": 0.33} },
    "stop_loss": { "price": 42.50, "type": "limit_order" }
  }
}
```

6.3 — The EGX Trading Constitution (15 Clauses)

File: tradingagents/agents/managers/risk_manager.py

The Constitution is based on Bai et al. (2022) Constitutional AI — explicit rule lists anchor LLM critiques better than vague "be safe" instructions.

Key clauses:

Clause 1: LONG-ONLY – no short selling on EGX
Clause 2: NO LEVERAGE – 100% cash positions only
Clause 3: $\pm 10\%$ daily price bands (List B) / $\pm 20\%$ (List A)
Clause 5: NO MARKET ORDERS – limit/VWAP/TWAP only
Clause 6: MAGNET ZONE – avoid entries within 1.5% of daily band
(Farag 2013, 2015: prices magnetically approach limits)
Clause 7: 1-day reversal after limit-down event
Clause 10: Max 10% per stock (UCITS Art. 52 compliance)
Clause 11: All trades need defined stop-loss (2×ATR primary, 5% fallback)
Clause 12: Daily entry $\leq 10\%$ ADV; throttle at 5-10% ADV
Clause 13: Max 2% of portfolio at risk per trade (Elder 1993 Rule)

EGX_RISK_LIMITS dictionary (deterministic enforcement):

```
"max_single_stock_pct":    0.10    # UCITS Art. 52(2)
"throttle_adv_threshold":  0.05    # WARN + THROTTLE at 5% ADV
"max_position_vs_adv_pct": 0.10    # VETO at 10% ADV (hard cap)
"max_single_trade_loss_pct": 0.02   # Elder 2% rule
"daily_price_limit":       0.10    #  $\pm 10\%$  EGX circuit breaker
"magnet_zone_pct":         0.015   # Farag (2013)
"no_short_selling":        True
"no_leverage":             True
```

6.4 — Confidence Aggregation

File: tradingagents/graph/propagation.py

Quorum rule: Need ≥ 2 of 3 directional analysts (technical, fundamental, news) to have valid confidence scores. If only 1 available → overall_status="INSUFFICIENT_DATA" → trade rejected.

Weakest-link dampening:

```
# 30% weight on weakest signal, 70% on average
overall = 0.30 × min(confidences) + 0.70 × mean(confidences)

# Example:
# Tech=0.90, Fund=0.50, News=0.80
# overall = 0.30 × 0.50 + 0.70 × 0.733 = 0.15 + 0.513 = 0.66
# (not a naive average of 0.73)
```

Why weakest-link?

"If one analyst is very uncertain, the system should be more conservative overall. A 90%-confident technical signal doesn't override a 50%-confident fundamental signal — they both count, but the weak one drags down the overall."

Final position size:

```
LLM recommendation (shares)
  × overall_confidence      ← from propagation
  × position_size_multiplier ← from sentiment blend
  × RL size_multiplier      ← from CQL policy (if trained)
all subject to ADV cap and portfolio cap
```

6.5 — Signal Processing

File: tradingagents/graph/signal_processing.py

The final BUY/SELL/HOLD is extracted by REGEX, not LLM:

```
Priority order:
1. "VETO" anywhere → HOLD
2. JSON "action" or "decision" field → extract value
3. "FINAL TRANSACTION PROPOSAL: BUY" marker → extract
4. First bare BUY / SELL / HOLD word → extract
5. Fallback → HOLD
```

Why regex?

"No LLM call = no hallucination risk, 100% deterministic, auditable. The LLM has already made its decision; we just need to parse it out."

7. Master Q&A — Hard Technical Questions

Social Media

Q: Why not just use ChatGPT to read posts and say if it's bullish/bearish?

"Two problems: (1) LLMs give inconsistent numeric scores across runs — same post, different score tomorrow. Our FinBERT/CAMeLBERT pipeline gives the same score every time. (2) LLMs don't understand Egyptian dialect Arabic well. CAMeLBERT was specifically trained on dialectal Arabic including Egyptian colloquial."

Q: What happens if Facebook blocks the Apify scraper?

"We have a stale-cache fallback — if Apify returns 402/403/429/5xx, we serve the last successful scrape (from DiskCache with 1-hour TTL). For completely stale data, the pipeline returns NO_SIGNAL with an honest reason code. The system degrades gracefully rather than crashing."

Q: How do you handle code-switching (Arabic + English in one post)?

"XLM-R handles that. It was trained on multilingual data including code-switched social media text. The language detector checks character ratios; if neither pure Arabic nor pure English, XLM-R gets the text."

Q: The aggregation weights engagement using log — why not linear?

"Linear weighting would let a single viral post with 10,000 likes dominate the signal over 100 posts with 10 likes each. Log dampens the amplification — post influence grows with engagement but at a decreasing rate. This prevents manipulation by artificially boosted posts."

Q: What's the backtest honesty gate in the social pipeline?

"Social APIs return CURRENT posts, not historical ones. If you backtest 2023 using today's Facebook data, you'd be injecting future sentiment into 2023 decisions — data leakage. Our `signal_adapter.py` detects if the requested date is >1 day in the past and either (a) replays archived posts from Postgres, or (b) returns NO_SIGNAL if no archive exists."

Memory

Q: Why not just use a standard SQL database for memory?

"SQL supports exact-match and keyword search. We need semantic similarity — 'stock approaching resistance, volume spike' should retrieve memories about 'price near 52-week high

with elevated volume' even though the words differ. That requires vector embeddings and cosine similarity."

Q: What's the minimum similarity threshold and why 0.30?

"0.30 means we only retrieve memories where the current situation is at least 30% cosine-similar to a past situation. Below that, the memory is more noise than signal. Too high (e.g., 0.90) and you'd almost never find relevant past situations. 0.30 is empirically the 'better than random' floor."

Q: What if ChromaDB is empty (first run)?

"We seed it with 41 hand-curated EGX precedent scenarios — real situations like 'sector-wide correction during political uncertainty' or 'breakout above 52-week high on earnings beat.' These give agents useful context from day 1 without requiring prior trading history."

Reinforcement Learning

Q: Why offline RL? Why not online RL?

"Online RL requires a simulation environment where the agent takes actions and receives rewards in real-time. We'd need a full EGX market simulator which is itself a major research project. We have a fixed historical dataset — offline RL is the principled approach."

Q: Why CQL not PPO or SAC?

"PPO and SAC require online interaction. For purely offline data, the standard choices are CQL, IQL, or TD3+BC. CQL is specifically designed to prevent Q-value overestimation on unobserved actions — the key risk with small datasets."

Q: 5,000 parameters is extremely small. Why?

"We have roughly 1,500 training samples. A larger network would overfit catastrophically. The 5,000-parameter network has just enough capacity to learn regime-aware sizing patterns without memorizing individual trade outcomes."

Q: What's the risk-free rate in your Sharpe ratio and why?

"24% — the Egyptian CBE policy rate during 2024-2026. Using the US risk-free rate (5%) would dramatically overstate Sharpe. EGX returns must be measured against the local opportunity cost of money (government T-bills). Standard practice for EM markets."

Q: How do you prevent RL from learning look-ahead bias?

"All rewards are computed from raw OHLCV data with a hard assertion that `trade_date < forward_window_start_date` . Any record where the 20-day forward window hasn't completed is marked `reward=None` and dropped before training."

Database

Q: Why 5 databases? Isn't that over-engineered?

"Each does something the others can't efficiently: PostgreSQL for ACID audit, ChromaDB for vector similarity (HNSW/IVFFlat indexing), Redis for sub-millisecond pub/sub, DiskCache for TTL-expiring API cache without wasting RAM, BM25 as an in-memory fallback requiring zero external services."

Q: What happens if Postgres goes down?

"The analysis pipeline continues — audit writes are skipped with a warning. The system degrades from 'full audit mode' to 'stateless analysis mode.' The decision is still made and returned to the user."

Q: How do you prevent duplicate social media posts?

"Each post gets `post_hash = SHA256(platform|url|timestamp|text[:500])` . The table has a UNIQUE constraint with `ON CONFLICT DO NOTHING` . Re-scraping the same source is safe — duplicates are silently ignored."

Portfolio Manager

Q: Why not let the LLM decide position size directly?

"Position sizing arithmetic is unreliable from LLMs — they have no access to real-time ADV data or portfolio state. Deterministic limits (ADV cap, portfolio cap) are computed in Python and explicitly override anything the LLM suggests. The LLM handles logic; Python handles math."

Q: What's the magnet effect and why does it matter for EGX?

"Farag (2013, 2015) documented that EGX stock prices magnetically approach the $\pm 10\%$ daily limit — once within 1.5%, market participants expect the limit to hit and it becomes self-fulfilling. Clause 6 prevents entering positions in this zone."

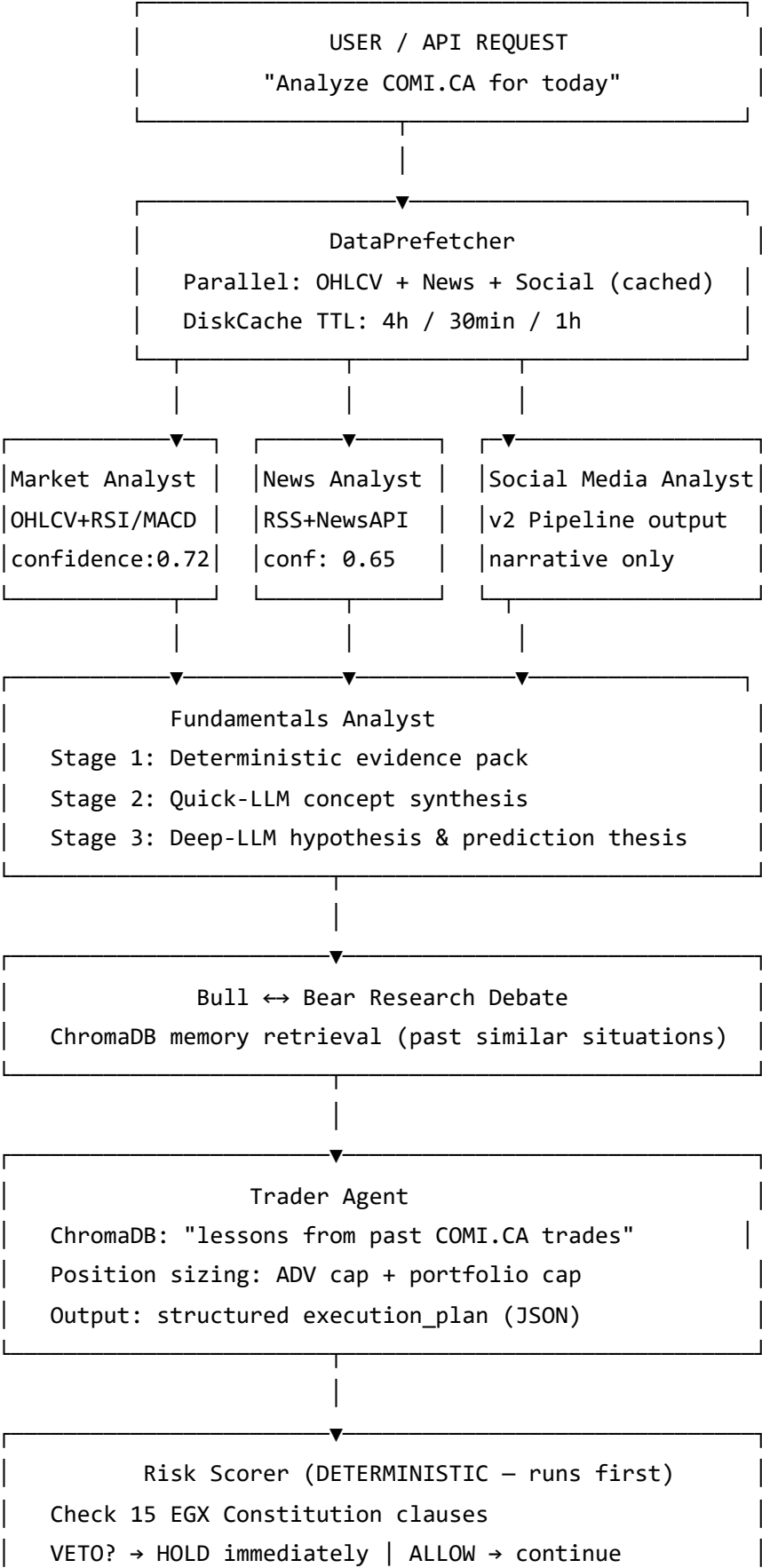
Q: What does the weakest-link dampening prevent?

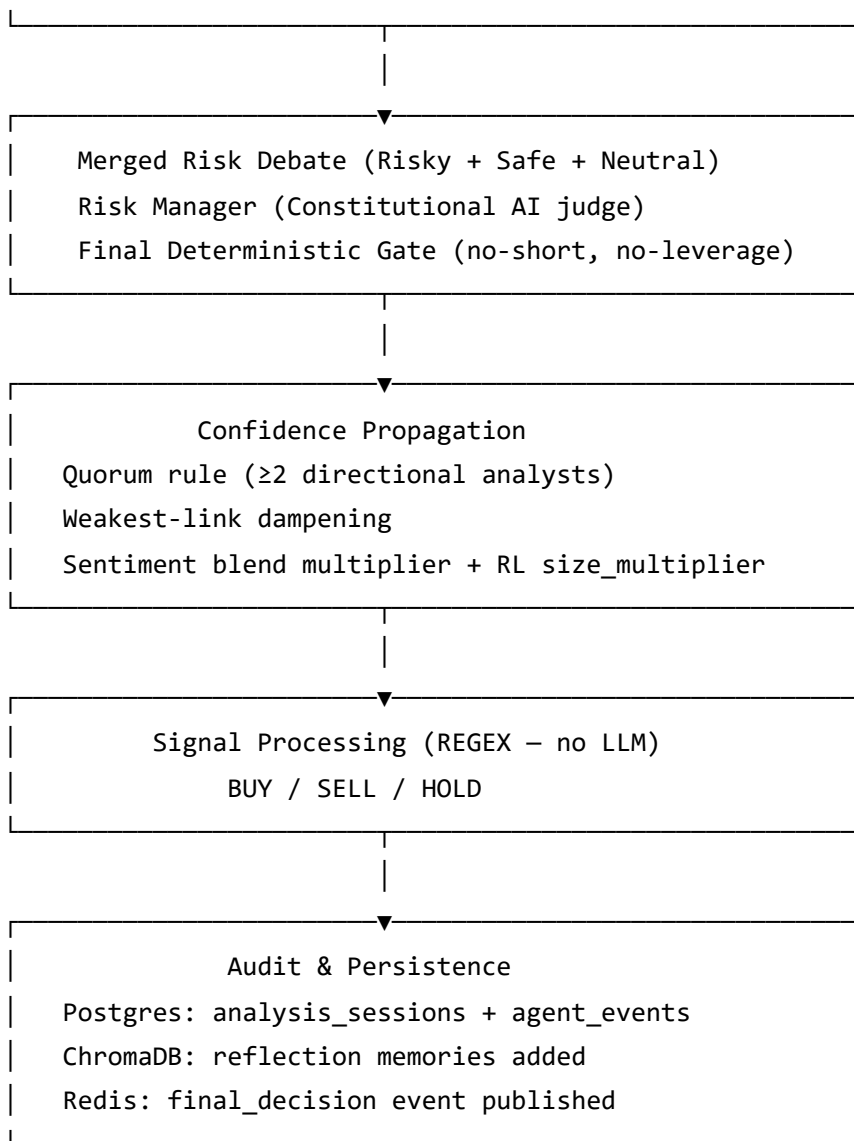
"It prevents over-confidence when analysts disagree. Naive average of 90%, 50%, 80% = 73%. With weakest-link: $0.3 \times 50\% + 0.7 \times 73\% = 66\%$. The 50% fundamental confidence drags down the result appropriately."

Q: Why is position sizing different in backtest vs live mode?

"UCITS Art. 52(2) requires $\leq 10\%$ per stock in live trading. In single-ticker backtests, enforcing this caps you at 10% of capital and distorts returns. Backtest mode removes the cap to evaluate signal quality in isolation. The flag must be False for any live deployment."

VISUAL: Full System Data Flow





KEY LITERATURE TO CITE

Topic	Reference
Constitutional AI (Risk Constitution)	Bai et al. (2022)
Conservative Q-Learning (CQL)	Kumar et al. (2020)
ATR stop-loss	Wilder (1978); Kaminski & Lo (2014)
EGX magnet effect	Farag (2013, 2015)
2% per-trade risk rule	Elder (1993)

Topic	Reference
Market impact / ADV	Almgren & Chriss (2000)
LLM judge bias mitigation	Zheng et al. (2023)
Multi-agent LLM trading	FinCon, Yu et al. NeurIPS (2024)
L-VaR liquidity decomposition	Bangia et al. (1999)
Safe RL shielding	Alshiekh et al. (2018)
UCITS position limits	UCITS Directive Art. 52(2)
CAMeLBERT Arabic NLP	CAMeL-Lab (2021)

ONE-LINE SUMMARIES (memorize these)

Social media sentiment:

"A 3-layer system: a 7-stage deterministic ETL pipeline produces weighted scores, a 3-model transformer engine handles bilingual text, and an LLM agent synthesizes it into narrative without touching the scores."

Why agent + engine:

"The engine gives deterministic, auditable, reproducible scores. The agent gives human-readable context. Separating them means scores never depend on an LLM's inconsistency."

Memory:

"ChromaDB vector store (or BM25 fallback) enables semantic retrieval of past trade lessons. Each of 5 agent roles has its own collection, seeded with EGX precedents on cold start."

RL:

"Single-step Conservative Q-Learning on ~1,500 EGX samples. Learns to reduce (never grow) the LLM's position sizing based on market regime + analyst confidence. CQL prevents overestimation on unobserved actions."

Databases:

"5 stores, each chosen for a specific access pattern: Postgres for durable audit, ChromaDB for vector memory, Redis for real-time streaming, DiskCache for TTL API caching, BM25 for offline fallback."

Portfolio manager:

"Trader agent produces execution plans; deterministic risk scorer checks 15 EGX hard rules first; LLM risk debate adds nuance; final gate enforces long-only/no-leverage. Confidence uses weakest-link dampening and quorum rules."